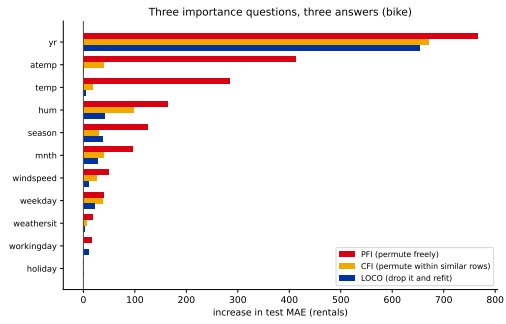
Lecture 23 told you to drop `atemp`. Should you?

Two measurements on the *same* bike model, from the interpretability chapter:

- **Permutation importance** ranks `atemp` **second** of eleven (413, behind `yr` at 766). Shuffle it and the model gets much worse.
- **Leave-One-Covariate-Out** says $-\mathbf{0.7}$. Delete the column, refit, and the model gets *very slightly better*.

So: do we drop it?



Neither number answers the question

- Permutation importance says **the model leans on it**.
- Leave-One-Covariate-Out says **the model copes without it**.

Both are true. `temp` carries almost the same signal, so removing `atemp` costs nothing — that is the "correlated features split the credit" result from lecture 22, seen from a different angle.

A ranking tells you the order.
It does not tell you where to cut.

Every tool in chapter 5 — permutation importance, SHAP, Lasso — produces an ordered list. Not one of them produces a *subset*. That gap is this entire lecture.

And now you have 230 of them

Last lecture ended by mechanically generating **230 candidate features** from the original eleven, in about thirty lines of code.

Ranking 230 features is easy — one line of `sklearn`. Deciding which *fifteen* go into production is not. That is the second half of last lecture's thesis: *generate cheaply*, **select ruthlessly**.

Outline

Fewer features, on purpose

Five good reasons — and one honest reason not to bother.

Five reasons to drop features

1. **The curse of dimensionality** (math module 30). Volume grows exponentially with p; with n fixed, your data gets sparser with every column you add.
2. **Multicollinearity** (lecture 22). Correlated features make coefficients unstable — exactly the `temp`/`atemp` problem from slide 1.
3. **Variance.** More features means more flexibility means more overfitting. Regularization helps; it does not eliminate.
4. **Compute and latency.** Every feature must be computed, stored, and served — forever, for every prediction, including the useless ones.
5. **Interpretability and audit.** "We use 8 features" defends far better than "we use 800". In credit, healthcare and insurance this is not optional.

Reason 4 is the one students underrate. A feature is not free once it is chosen — somebody maintains that pipeline at 3am when the upstream source changes.

Predict-first: does adding features always help?

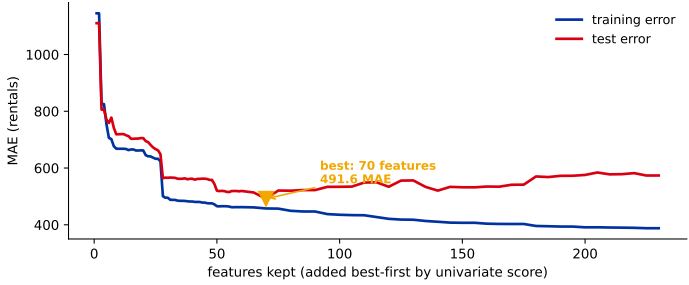
You add candidates one at a time, best-first, refitting Ridge after each.

What happens to (a) the training error, (b) the test error?

Predict-first: does adding features always help?

You add candidates one at a time, best-first, refitting Ridge after each.

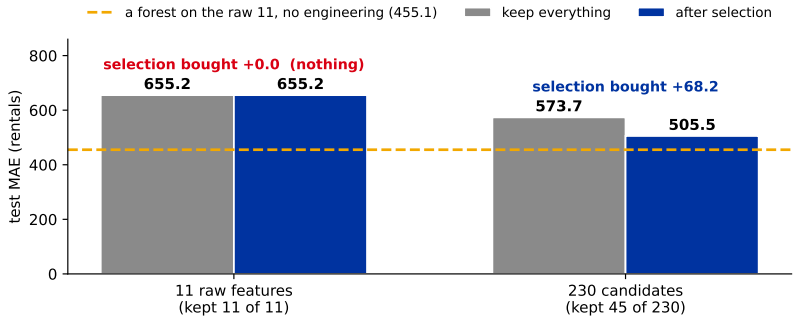
What happens to (a) the training error, (b) the test error?



Training error falls forever — more columns can only fit the training set better. **Test error is U-shaped**: down while features carry signal, then up as they bring more variance than they remove bias. Selection is the search for that minimum.

So how much does selection actually buy?

Two regimes, same data, same model, same split:



Eleven honest features. Recursive elimination with cross-validation kept **all eleven**. Selection bought **exactly nothing** — not "a little", *zero*.

Two hundred and thirty candidates. It kept **45** and bought +**68.2** MAE.

When *not* to select

Selection pays in proportion to how much junk you generated. If you did not mass-produce candidates, you probably do not need this lecture's machinery — fit the model on everything and regularize.

Three cases where selection is not worth it:

- p **is small and every feature was chosen deliberately.** The bike eleven.
- **You are already regularizing hard.** Ridge and Lasso select *softly* (lecture 7); a hard cut can only lose the small-but-real signal they were keeping.
- $n \gg p$**.** With plenty of rows, the variance cost of a weak feature is small.

And note the orange line on the previous slide. Engineering 219 features and selecting the best 45 got Ridge to 505.5. A **random forest on the raw eleven**, with no feature engineering and no selection at all, gets **455.1**. Sometimes the answer is "change the model", not "change the features".

You have done this before

without calling it selection.

Refresher: what each chapter-5 tool measures

Tool	The question it answers	Fro
Permutation feature importance (PFI)	how much does the model *rely* on it?	L23
Conditional feature importance (CFI)	. . . beyond what its correlated partners give?	L23
Leave-One-Covariate-Out (LOCO)	how much worse if I *delete* it and refit?	L23
Shapley Additive Global importance (SAGE)	what is its fair share of the performance?	L23
SHapley Additive exPlanations (SHAP)	how much did it move *this* prediction?	L24
Lasso coefficient path	is it worth a non-zero coefficient?	L7,

Six tools, six different questions — and **six ordered lists**. Slide 1 is what happens when two of them disagree: they were never answering the same question.

Read them again as selection algorithms

- **Leave-One-Covariate-Out is one step of backward elimination.** Drop a feature, refit, measure. You have already run it. Repeat it p times greedily and you have *recursive feature elimination* (section 4).
- **The Lasso path is embedded selection.** A coefficient hitting exactly zero *is* the feature being dropped. You have watched this happen since lecture 7.
- **Impurity and permutation importance are the input** that wrapper methods consume — they decide *which* feature to drop next.
- **Boruta's shadow features** (section 4) are permutation importance with a null distribution bolted on: instead of "how important?", ask "more important than a *shuffled copy* of itself?"

You are not starting from zero. You are missing exactly three things — and the next slides are those three.

Two different goals, and students conflate them

Minimal-optimal

The *smallest* subset that predicts as well as the full set.

- Lasso, recursive elimination, stability selection
- correlated duplicates get dropped *arbitrarily*
- what you want when you **ship a model**

All-relevant

Every feature carrying signal about y, duplicates included.

- Boruta
- keeps both members of a correlated pair
- what you want when you are **doing science**

Bike makes this concrete, and it finally answers slide 1. Minimal-optimal *drops* `atemp` — given `temp`, it adds nothing. All-relevant *keeps* it — it genuinely predicts bike rentals. **Same data, both answers correct, different question.** You cannot pick a method until you know which one you are asking.

The distinction is due to Nilsson et al. (2007).

So what is actually left to learn?

A ranking gives you an order. It does not give you:

1. **A cutoff.** Where does the list stop? *(sections 3 and 4)*
2. **A way to score a subset rather than a feature.** Features interact; the best three individually are rarely the best three together. *(section 4)*
3. **A guarantee the answer is stable.** Run it on a slightly different sample — do you get the same list? *(section 5)*

Three families of methods map onto these: **filter** (cheap, one feature at a time), **embedded** (free with the model — that is chapter 5), and **wrapper** (expensive, scores subsets). We spend the rest of the lecture on filter and wrapper, because you already have embedded.

Score every feature once

Before you fit anything at all.

The three families

Family	How it works	Trade-off
Filter	score each feature *independently* against y, keep the top k	fast, model-agnostic. **Blind to interactions.**
Embedded	the model selects *while* fitting (Lasso, tree importances)	cheap, model-aware. Tied to that one model.
Wrapper	search over *subsets*, score each by cross-validation	most accurate. Refits the model $\mathcal{O}(p)$ times.

The standard funnel — cost rises left to right, so spend it late:

$$10{,}000 \xrightarrow[\text{filter}]{} 500 \xrightarrow[\text{embedded}]{} 50 \xrightarrow[\text{wrapper}]{} 15$$

The canonical survey is Guyon & Elisseeff (2003), still worth reading.

Variance thresholds and univariate filters

Step 1: drop the near-constants. A column that never varies cannot predict anything. Free, and always worth doing.

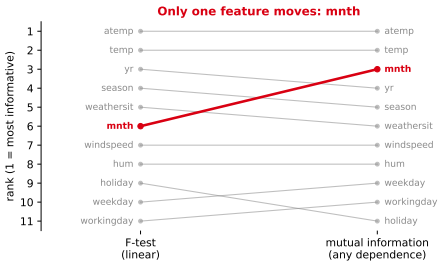
Step 2: score each feature against y.

- **Regression:** `f_regression` (linear association only), `mutual_info_regression` (*any* dependence)
- **Classification:** `f_classif`, `chi2`, `mutual_info_classif`

```
X = VarianceThreshold(0.01).
    fit_transform(X)

# top 20 by linear association
SelectKBest(f_regression, k=20).
    fit(X, y)

# top 20 by ANY dependence
SelectKBest(mutual_info_regression
    ,
            k=20).fit(X, y)
```

The choice between those two scores is not cosmetic. `f_regression` measures *linear* association only — it is blind to exactly the effects lecture 26 spent a slide binning.

Where the two scores disagree



Ten of the eleven features rank almost identically under both scores. **One moves three places:** `mnth`**.**

F-test rank **6** $\rightarrow$ mutual-information rank **3**.

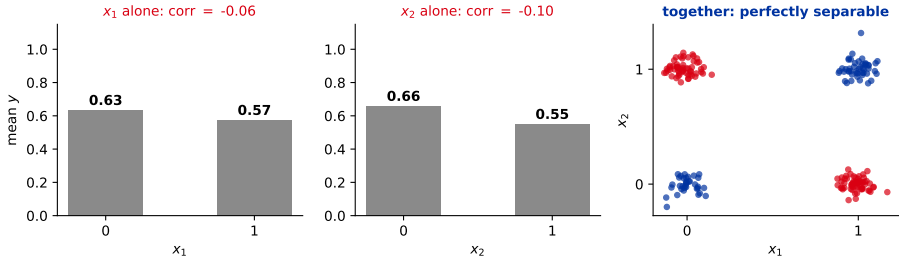
Why: month is **cyclic**. Rentals are low in January, high in July, low again in December, so the *linear* association is weak while the actual dependence is strong. The F-test cannot see a shape it cannot draw with a straight line.

This is exactly what the cyclic sin / cos encoding from lecture 3 exists to fix.

If you had filtered on `f_regression` alone at $k = 5$, you would have thrown away `mnth` — the third most informative column you had.

The independence trap

Filters score each feature *alone*. Consider two binary features with $y = x_1 \oplus x_2$:



Each feature alone: **no signal**. Mean y is the same whether x_1 is 0 or 1. Correlation ≈ 0, mutual information ≈ 0.

Together: **a perfect predictor.** Any filter, univariate or not, deletes both features before the model ever sees them.

A filter is a screen, not a decision. Use it to go from 10,000 to 500, never from 500 to 15. A feature that fails on its own may be indispensable in company.

Multivariate filters

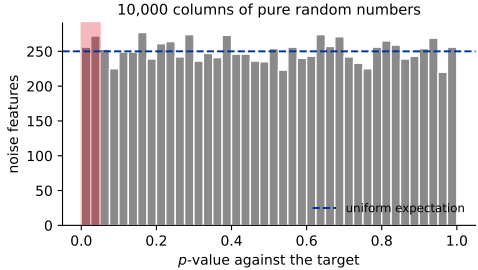
Filters that look at features *together*, without the cost of a wrapper:

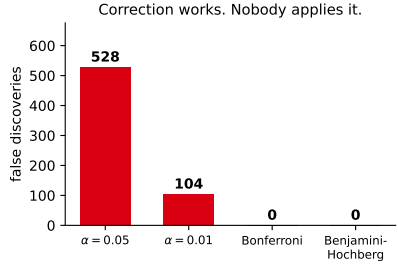
- **Minimum Redundancy Maximum Relevance** (mRMR). Greedily pick the feature maximising (relevance to y) − (mean correlation with what you already picked). Directly attacks the `temp`/`atemp` problem: the second one adds relevance but also redundancy.
- **Cluster-and-represent.** Hierarchically cluster the features by correlation, keep one representative per cluster. Crude, cheap, effective when p is huge.
- **Hilbert-Schmidt Independence Criterion Lasso** (HSIC-Lasso). Kernel-based, catches non-linear dependence, scales to many thousands of features.

Libraries: `mrmr-selection`, `scikit-feature`. These are not the main event in most projects, but they earn their keep in genomics and sensor data, where p is in the tens of thousands and half the columns are near-duplicates.

How many features pass by pure chance?

Score **10,000 columns of random numbers** against the bike target:





528 clear $\alpha = 0.05$. The best random column reaches $p = 1.17 \times 10^{-4}$ — a number most people would call significant without blinking.

The *p*-values are *uniform*, exactly as theory says. 5% of them land below 0.05 because that is what 0.05 *means*.

Control the family-wise error rate with **Bonferroni** (α/m) or the false discovery rate with **Benjamini-Hochberg** (1995) — `SelectFwe`, `SelectFdr`. Both remove all 528. **Almost nobody applies them to a feature pool**, and last lecture's 230 mechanical candidates are exactly this problem at smaller scale.

Stop scoring features

Start scoring subsets.

Recursive Feature Elimination

The algorithm. Fit the model. Rank the features. Drop the weakest. *Refit.* Repeat.

Why the refit matters. Importances are not fixed — they change as features leave. Drop `atemp` and `temp`'s importance *jumps*, because it is no longer splitting the credit with a near-duplicate (lecture 22).

A one-shot ranking cannot see that. Recursive elimination re-measures after every removal, which is exactly why it beats "keep the top k of a single ranking".

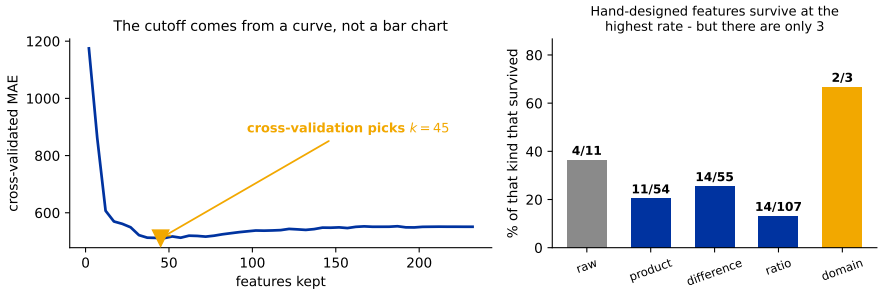
```
from sklearn.feature_selection
    import RFE

rfe = RFE(Ridge(),
          n_features_to_select=15,
          step=1).fit(X, y)
X_sel = X[:, rfe.support_]
```

Cost: $\mathcal{O}(p)$ refits. Use a fast model, or pre-filter first.

Introduced for gene selection with support vector machines by Guyon et al. (2002), where p was in the thousands and n in the dozens.

Where the cutoff finally comes from

`RFECV` cross-validates *at every subset size* and keeps the best one. The cutoff is no longer your judgement call — it is a measurement.



On the 230 candidates it chose **k = 45**. **This slide answers slide 2:** you do not read the cutoff off a bar chart, you find it with cross-validation. *Hand-designed* features survived at the highest rate of any kind (2 of 3, versus 13% of the ratios) — but there were only three of them to begin with.

Sequential selection: forward and backward

Forward — start empty, repeatedly add the feature that most improves cross-validation. Fast when you want *few* features.

Backward — start full, repeatedly drop the one whose removal hurts least. Fast when you want to drop only a few. *This is Leave-One-Covariate-Out, run greedily to exhaustion.*

```
SequentialFeatureSelector(
    Ridge(),
    n_features_to_select=15,
    direction="forward",    # or "
        backward"
    cv=5).fit(X, y)
```

Both are greedy, and greedy fails on the XOR trap. Forward selection evaluates x_1 alone (useless), then x_2 alone (useless), and never adds either — so it never discovers that together they are perfect. Backward selection has the mirror problem. When features only pay off jointly, *no* greedy method finds them.

Boruta: asking a different question

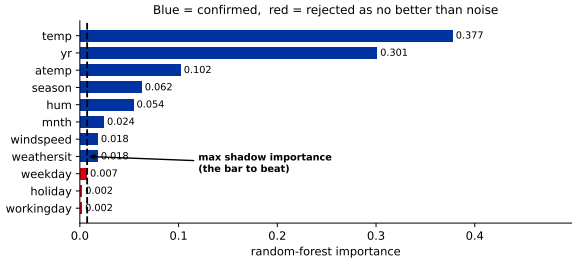
Every method so far asks "which features are *best*?" Boruta asks **"which features are better than nothing?"**

The algorithm (Kursa & Rudnicki, 2010):

1. For every real feature, add a **shadow** — a shuffled copy of that same column. It has the same distribution and, by construction, *no relationship to y*.
2. Fit a random forest on real + shadow features together.
3. Any real feature scoring above the **maximum** shadow importance gets a hit. (The maximum, not the mean — this is the multiple-testing correction from slide 20, built in.)
4. Repeat with fresh shuffles. Over many iterations, a feature that consistently beats the best shadow is **confirmed**; one that consistently loses is **rejected**.

This is the **all-relevant** method from slide 12. It is not trying to find a small set — it is trying to find *every* column that carries signal.

Boruta on bike — and a flat contradiction



Both `temp` and `atemp` confirmed — as slide 12 predicted, all-relevant keeps the correlated pair that recursive elimination throws half of away.

But look at the bottom three. Boruta *rejects* `holiday`, `weekday` and `workingday`: no better than a shuffled column.

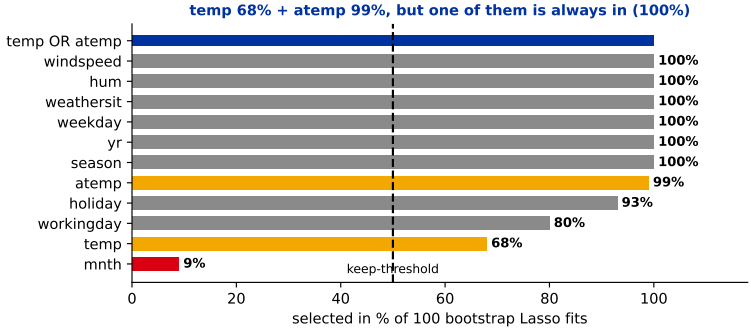
And the next slide will show Lasso keeping `weekday` in 100% of bootstrap samples. Same eleven features, same data, opposite verdicts. Not a bug: Boruta asks "does this beat noise *inside a forest*?" while Lasso asks "does this earn a coefficient in a *linear* model?" A weekday effect a tree already absorbs through `season` and `temp` can still be worth its own coefficient.

Run it twice

Do you get the same features?

Stability selection

One Lasso fit gives one answer. Fit it on **100 bootstrap resamples** and count how often each feature survives (Meinshausen & Bühlmann, 2010):



`atemp` **99%,** `temp` **68% — but one of the two is selected in 100% of resamples.** They are not splitting the credit evenly; `atemp` is marginally the better predictor, so Lasso reaches for it first and `temp` only gets in when the resample happens to favour it. **Correlated features split credit in proportion to a small edge**, and the weaker one then looks dispensable when it is not.

What stability tells you that a single fit cannot

The pair

Report the *union*, not the members. "One temperature column, always" is the true finding; "`temp` is borderline at 68%" is an artefact of having two.

The genuinely unstable one

`mnth` survives in **9%** of resamples — it flickers in and out. And it is the same column that the F-test undersold on slide 17. A feature a *linear* model can barely use is exactly the one a *linear* selector cannot make its mind up about.

Stability selection also answers *how many* for free: k is however many features clear your threshold. No elbow to eyeball, no k to guess.

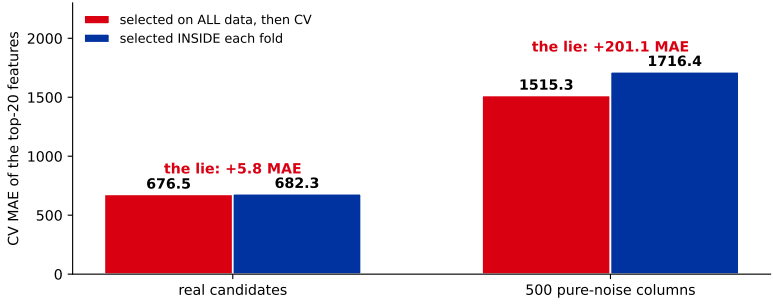
Four ways to pick k, in practice: the `RFECV` curve, a stability threshold, Boruta's shadow cutoff — and **a hard business constraint**. "The credit model ships with twelve features because a regulator reads all of them" is a real reason, and it overrules all three of the others.

The mistake that makes everything look great

And it is the same mistake as last lecture's, one level up.

Selection bias: selecting outside the loop

Rank all 230 features on the **whole dataset**, keep the top 20, *then* cross-validate them. The selection already saw the test rows.



On real candidates the lie is worth +5.8 MAE — unpleasant but survivable. On **500 columns of pure noise** it is worth +**201.1**, and the model looks respectable *on data containing no signal whatsoever*. **The bias is worst exactly when you have the most junk** — which is to say, right after last lecture.

The fix is the same rule as lecture 6

Cross-validation estimates the performance of a **procedure**, not of a fitted model. If selection is part of your procedure, it goes *inside* the loop.

```
# WRONG - selection saw every row before the split
X_sel = SelectKBest(f_regression, k=20).fit_transform(X, y)
cross_val_score(Ridge(), X_sel, y, cv=5)

# RIGHT - selection is refitted inside every fold
pipe = Pipeline([("sel", SelectKBest(f_regression, k=20)),
                 ("sc",  StandardScaler()),
                 ("m",   Ridge())])
cross_val_score(pipe, X, y, cv=5)
```

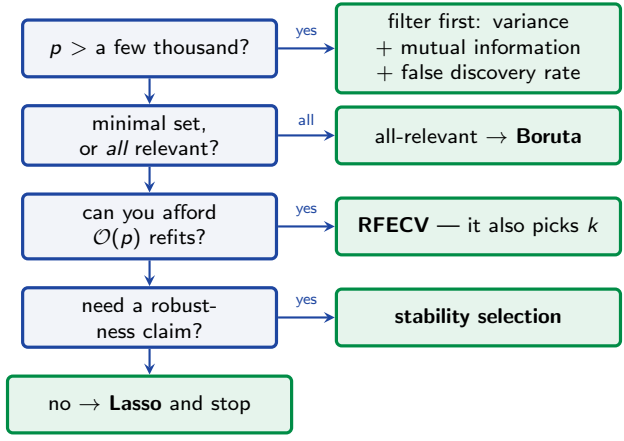
Identical in shape to lecture 26's rule for engineered features: **if it learns anything from the data, it belongs inside the** `Pipeline`**.** A selector learns *which columns to keep* — that is learning, and it leaks.

And if you tune hyperparameters *and* select features, you need **nested** cross-validation — an outer loop for the estimate, an inner one for the choices.

Four more pitfalls

1. **Leakage through the encoder.** If a feature was target-encoded without out-of-fold discipline (lecture 26), selection just ranks the leak highest. Fix the encoder before you trust the ranking.
2. **Model-dependence.** Lasso's picks are not the forest's picks — slide 26 is a whole slide about that. **Selection results do not transfer across model classes.** Change the model, redo the selection.
3. **Arbitrary thresholds.** "Keep importance > 0.01" is a number somebody made up. Use a cross-validated curve, a stability frequency, or a shadow comparison — all three are defensible, and 0.01 is not.
4. **Forgetting the baseline.** Always compare against keeping everything. Slide 8 did exactly this and found selection was worth *zero* on the raw eleven. **Selection that does not beat keep-all was not worth doing.**

Which method, when?



Whichever branch you take: do the selection **inside** the cross-validation loop, and compare against keeping everything.

Recap

- **A ranking is not a decision.** Chapter 5's tools order features; none of them picks a cutoff. That is what filter and wrapper methods are for.
- **Selection pays in proportion to the junk you generated:** +0.0 MAE on the honest eleven, +68.2 on 230 mechanical candidates.
- **Filter is a screen, not a decision** — fast, but blind to interactions (XOR) and to non-linear effects (`mnth`, rank 6 by F-test and rank 3 by mutual information).
- **Wrappers score subsets.** `RFECV` finds the cutoff by cross-validation; it chose $k = 45$. Greedy sequential methods still miss jointly-useful pairs.
- **Minimal-optimal $\neq$ all-relevant.** Recursive elimination drops `atemp`; Boruta keeps it. **Both are right** — so answer slide 1 by naming your question.
- **Check stability.** `atemp` 99%, `temp` 68%, but one of them is in 100% of resamples. Report the union.
- **Select inside the cross-validation loop.** Outside it, 500 pure-noise columns produced a respectable-looking model.

Next: classic methods — k-nearest neighbours, naive Bayes, discriminant analysis, support vector machines and Gaussian processes. Models where, unlike trees, the features you feed in are the features it gets.